Phishing URL Detection Using Machine Learning (Mini SOC Tool)

(SOC Analytics Tool)

Project Report

Submitted by:
Ritik Maurya

Intern ID:
2081

Domain:
Cybersecurity & Machine Learning

Technology Stack:
Python, Scikit-learn, Streamlit

Source Code (GitHub):

https://github.com/ritmaurya91-a11y/phishing-url-detector

1. Introduction

In modern IT infrastructures, system and authentication logs play a crucial role in monitoring user activities, detecting security incidents, and maintaining overall system integrity. Logs generated by operating systems, servers, and applications capture detailed information about login attempts, access behavior, and system events. However, the exponential growth of log data makes manual inspection inefficient, time-consuming, and prone to human error.

Security Operations Centers (SOCs) rely heavily on automated log analysis tools to detect suspicious activities such as brute-force attacks, unauthorized access attempts, and insider threats. Traditional rule-based detection mechanisms depend on predefined thresholds and static signatures, which limits their ability to identify unknown or evolving attack patterns.

Machine learning-based anomaly detection overcomes these limitations by learning normal system behavior and automatically identifying deviations without requiring labeled attack data. Unsupervised learning techniques are particularly suitable for log analysis, as real-world logs often lack accurate labels.

This project focuses on the design and implementation of a **Log Anomaly Detection System** using machine learning techniques. The system analyzes authentication logs and detects abnormal login behavior using the **Isolation Forest** algorithm. A **Streamlit-based web interface** is developed to simulate a **Mini SOC Analytics Tool**, enabling security analysts to upload logs and instantly identify suspicious events.

2. Objectives

The primary objectives of this project are:

1. **To detect anomalous login behavior from system logs**
   Identify abnormal authentication patterns such as repeated failed login attempts using machine learning.

2. **To automate log parsing and preprocessing**
   Convert raw authentication logs into structured and meaningful numerical features.

3. **To apply unsupervised machine learning techniques**
   Use Isolation Forest to detect anomalies without relying on labeled attack datasets.

4. **To provide a user-friendly SOC analytics interface**
   Enable log upload, anomaly detection, and result visualization through a web-based dashboard.

5. **To simulate a real-world SOC workflow**
   Replicate how SOC analysts monitor logs and flag high-risk authentication events.

3. Dataset Description

The dataset used for this project is derived from authentication and system log records, similar to those generated in real-world operating systems and enterprise environments. These logs resemble data collected from sources such as Linux authentication logs (`/var/log/auth.log`), Windows Security Event Logs, and SSH login records. Such datasets are widely used in cybersecurity research and SOC operations for analyzing user authentication behavior and detecting potential security threats.

Authentication logs capture detailed information about login attempts, including timestamps, user identities, and authentication outcomes. By analyzing patterns within these logs, security analysts can identify abnormal behavior such as brute-force attacks, credential misuse, and unauthorized access attempts.

For this project, the dataset is provided in CSV format and uploaded by the user through the Streamlit-based SOC dashboard. The dataset represents a realistic simulation of authentication events commonly monitored in Security Operations Centers.

3.1 Dataset Structure

Each record in the dataset represents a single authentication event and contains the following fields:

- timestamp – Date and time of the login attempt
- user – Username involved in the authentication process
- action – Result of the login attempt (Accepted / Failed)

In addition to these raw fields, a derived feature is generated during preprocessing to support anomaly detection.

3.2 Target and Feature Encoding

Since this project uses unsupervised machine learning, the dataset does

not contain predefined attack labels. Instead, anomaly labels are generated by the model during prediction.

Derived Feature Encoding

- failed_attempts
  - `1` → Failed login attempt
  - `0` → Successful login attempt

Anomaly Output Labels (Model Generated)

- `1` → Normal behavior
- `-1` → Anomalous behavior

This encoding allows the Isolation Forest algorithm to distinguish between normal authentication activity and abnormal login patterns.

3.3 Feature Categories

The dataset features can be broadly categorized into the following group:

a) Authentication-Based Features

These features are derived directly from login activity and authentication outcomes:

- Login success or failure status
- Frequency of failed login attempts
- Repeated authentication failures for the same user

These features are critical for SOC analysis, as repeated failed authentication attempts are a strong indicator of brute-force attacks and account compromise attempts.

3.4 Feature Selection for This Project

Although real-world authentication logs may contain additional information such as IP addresses, device identifiers, and geolocation

data, this project focuses on authentication-based features to ensure:

- Real-time anomaly detection
- Simplicity and clarity
- Fast processing of log data
- Lightweight deployment
- Easy understanding for SOC analysts

The primary feature used for model training and detection is:

1. failed_attempts

This feature effectively captures abnormal login behavior without requiring complex log enrichment.

3.5 Data Preprocessing

Before applying the machine learning model, the following preprocessing steps were performed:

- Conversion of raw CSV log data into a structured format
- Encoding of authentication results into numerical values
- Handling of missing or inconsistent entries
- Selection of relevant feature columns
- Preparation of feature vectors for anomaly detection

Since Isolation Forest is an unsupervised algorithm, the entire dataset is used for learning normal behavior patterns without splitting into labeled training and testing sets.

These preprocessing steps ensure consistency, reduce noise, and improve the reliability of anomaly detection results.

4. System Architecture

The SOC Log Anomaly Detection system is designed using a modular and layered architecture to ensure simplicity, scalability, and efficient real-time operation. The architecture follows a sequential processing pipeline, where each component performs a specific role in transforming raw authentication log data into meaningful security insights.

The system simulates a Mini Security Operations Center (SOC) analytics workflow, enabling automated detection of suspicious login behavior using machine learning techniques.

The overall workflow of the system is illustrated below:

User Uploads Authentication Logs (CSV)
↓
Log Parsing and Feature Extraction Module
↓
Machine Learning Anomaly Detection Model (Isolation Forest)
↓
Anomaly Detection and Decision Engine
↓
Streamlit Web Interface (SOC Dashboard)

4.1 User Interface Layer

The user interface layer is implemented using the Streamlit framework, which provides a lightweight and interactive web environment. This layer allows users or SOC analysts to upload authentication log files in CSV format and view anomaly detection results in real time.

The interface displays log details such as timestamp, username, authentication action, and anomaly status. This dashboard simulates the behavior of a simplified Security Operations Center (SOC), where analysts continuously monitor authentication activity and investigate suspicious login events.

4.2 Log Parsing and Feature Extraction Layer

The log parsing and feature extraction module processes the raw authentication logs uploaded by the user and converts them into numerical features suitable for machine learning analysis.

This layer extracts authentication-related indicators such as:

- Login success or failure status
- Encoding of failed login attempts
- Frequency and repetition of authentication failures

These extracted features represent security-relevant indicators that are strongly associated with brute-force attacks, unauthorized access attempts, and abnormal login behavior.

4.3 Machine Learning Layer

This layer contains the trained Isolation Forest anomaly detection model. The model receives the extracted numerical features as input and learns the normal authentication behavior patterns present in the dataset.

Isolation Forest was selected due to its effectiveness in unsupervised anomaly detection, ability to handle large datasets, and suitability for identifying rare abnormal events without requiring labeled attack data.

The model is trained dynamically on the uploaded log data and immediately used to generate anomaly predictions.

4.4 Anomaly Detection and Decision Layer

Based on the anomaly scores generated by the Isolation Forest model, the system assigns a final classification to each log entry:

- 1 → Normal behavior
- −1 → Anomalous behavior

This layer flags suspicious authentication events and forwards the detection results to the user interface for visualization. The decision output helps SOC analysts quickly identify high-risk login activities that

require further investigation.

4.5 Deployment Layer

The complete system is packaged as a standalone Streamlit web application, which can be deployed locally or on cloud platforms. The lightweight design ensures minimal resource consumption while supporting real-time log analysis.

This layered architecture enables easy maintenance, future enhancements, and seamless integration with other cybersecurity tools such as SIEM platforms, intrusion detection systems (IDS), and SOC monitoring dashboards.

5. Feature Engineering

Feature engineering is a critical stage in machine learning systems, as the quality of extracted features directly influences the accuracy and reliability of the anomaly detection model. In log-based security analytics, carefully selected authentication features help capture abnormal user behavior patterns that may indicate security threats such as brute-force attacks or unauthorized access attempts.

In this project, feature engineering focuses on extracting **security-relevant indicators directly from authentication log data**. This approach enables fast and efficient analysis without requiring external threat intelligence sources or labeled attack data.

5.1 Selected Authentication-Based Features

The following feature is extracted automatically from each authentication log entry:

1. Failed Login Attempt Indicator (failed_attempts)

Repeated authentication failures are a strong indicator of malicious activity, such as brute-force login attacks. The system analyzes the authentication outcome and encodes failed login attempts numerically.

- **Value = 1 → Failed login attempt**
- **Value = 0 → Successful login attempt**

This feature allows the system to distinguish between normal authentication behavior and suspicious login patterns.

5.2 Feature Encoding

All extracted features are encoded numerically using the following scheme:

- `0` → Normal authentication behavior
- `1` → Failed authentication attempt

These numerical values are directly compatible with the Isolation Forest anomaly detection model and preserve interpretability for SOC analysts.

5.3 Benefits of Authentication-Based Feature Engineering

The selected feature engineering approach provides several advantages:

- Enables real-time anomaly detection
- Eliminates dependency on labeled attack data
- Reduces computational overhead
- Supports unsupervised machine learning
- Improves detection of brute-force attacks
- Simplifies integration with SOC workflows

6. Machine Learning Model

6. Machine Learning Model

The Log Anomaly Detection system utilizes an unsupervised machine learning approach to identify abnormal authentication behavior from system logs. Since real-world log data often lacks labeled attack

information, unsupervised learning is well-suited for detecting previously unseen or unknown threats.

After evaluating multiple anomaly detection techniques, the Isolation Forest algorithm was selected as the primary model for this project due to its effectiveness in identifying rare abnormal events and its scalability for large log datasets.

6.1 Model Selection

Isolation Forest is an ensemble-based anomaly detection algorithm that works by randomly partitioning data points using decision trees. Anomalous data points are isolated faster than normal data points because they differ significantly from the majority of the data.

The key advantages of using Isolation Forest for this application include:

- Effective detection of rare anomalies
- No requirement for labeled training data
- Scalability for large log datasets
- Low computational overhead
- Robust performance on tabular data

These characteristics make Isolation Forest particularly suitable for SOC log analytics.

6.2 Training Process

The training process is performed dynamically on the uploaded authentication logs. The following steps are applied:

1. Loading authentication log data from the uploaded CSV file
2. Extraction of numerical authentication features
3. Training the Isolation Forest model on login behavior
4. Learning baseline normal authentication patterns
5. Generating anomaly predictions for each log entry

Since Isolation Forest is an unsupervised model, the entire dataset is used for learning normal behavior without splitting into labeled training and testing sets.

6.3 Model Output and Interpretation

The trained Isolation Forest model assigns an anomaly label to each authentication event:

- 1 → Normal behavior
- -1 → Anomalous behavior

These outputs allow SOC analysts to quickly identify suspicious login attempts that may indicate brute-force attacks or unauthorized access.

6.4 Model Deployment

The trained Isolation Forest model is integrated directly into the Streamlit-based web application. The model is instantiated at runtime and processes uploaded log files instantly without requiring retraining for each prediction session.

This deployment strategy ensures:

- Real-time anomaly detection
- Fast inference
- Low memory usage
- Reproducible results
- Seamless integration with SOC dashboards

7. Web Application Implementation (Streamlit)

To provide a user-friendly interface for the phishing URL detection system, the project leverages **Streamlit**, a Python-based framework for building interactive web applications
quickly and efficiently. Streamlit allows users to input URLs, receive real-time predictions, and visualize confidence levels without requiring any installation beyond Python and the required libraries.

7.1 User Interface Design

7. Web Application Implementation (Streamlit)

To provide a user-friendly interface for the **SOC Log Anomaly Detection system**, the project leverages **Streamlit**, a Python-based framework for building interactive web applications quickly and efficiently. Streamlit enables security analysts to upload authentication log files, perform real-time anomaly detection, and visualize suspicious login behavior without requiring complex setup or external dependencies.

The web application simulates the workflow of a **Mini Security Operations Center (SOC) analytics tool**, allowing efficient inspection of authentication logs.

7.1 User Interface Design

The web interface is designed to be simple, intuitive, and functional, closely resembling a Mini SOC dashboard. The key components of the interface include:

1. Title and Description

○ Clearly indicates the purpose of the application:
“Log Anomaly Detection System (SOC Analytics Tool)”

○ Provides brief instructions to users on how to upload authentication log files for analysis

---

2. File Upload Component

○ Allows users to upload authentication log files in **CSV format**
○ Supports logs containing timestamps, usernames, and authentication actions

---

3. Analyze Button

○ Triggers log parsing and feature extraction
○ Initiates the machine learning–based anomaly detection process
○ Displays detection results instantly

---

4. Results Display

○ Shows authentication log entries along with anomaly classification
○ Clearly highlights suspicious login events
○ Enables SOC analysts to quickly identify high-risk activities

---

5. Visual Indicators

○ Anomalous log entries are highlighted for easy identification
○ Normal authentication behavior is displayed without alerts

7.2 Application Workflow

The web application follows a clear and sequential workflow:

1. User Input

○ The user uploads an authentication log file (CSV) using the file uploader

---

2. Log Parsing and Feature Extraction

○ Uploaded logs are processed to extract relevant authentication features
○ Failed and successful login attempts are encoded numerically

---

3. Anomaly Detection Engine

○ Extracted features are passed to the Isolation Forest model
○ The model analyzes authentication behavior and identifies anomalies

---

4. Output Rendering

○ Anomaly detection results are displayed in real time
○ Suspicious login attempts are clearly marked for analyst review

---

7.3 Technical Implementation

Python Libraries Used

- **Streamlit** – User interface and web application
- **Pandas** – Data preprocessing and log handling
- **Scikit-learn** – Isolation Forest machine learning model

File Structure

```
Log_Anomaly_Detection/
│
├── app.py                # Streamlit application
├── model.py              # Isolation Forest model logic
├── logs.csv              # Sample authentication logs
├── requirements.txt      # Project dependencies
└── report.pdf            # Project report
```

Deployment

○ The application can be executed locally using:

```
streamlit run app.py
```

○ The lightweight design allows deployment on cloud platforms such as **Streamlit Cloud, AWS, or Heroku**

7.4 Benefits of Streamlit Implementation

- Rapid prototyping and deployment
- Interactive and easy-to-use SOC dashboard
- Real-time log anomaly detection
- Minimal system resource usage
- Suitable for integration into Mini SOC environments

8. Results and Evaluation

This section presents the performance analysis of the phishing URL detection system after training and deploying the machine learning model. The evaluation focuses on accuracy, reliability, confidence scoring, and real-world testing using both legitimate and phishing URLs.

8.1 Model Performance Metrics

The Random Forest classifier was evaluated using a held-out test dataset. The following metrics were used to measure effectiveness:

- **Accuracy:** Measures overall correctness of predictions

● **Precision:** Percentage of URLs classified as phishing that are truly phishing ●

Recall (Detection Rate): Percentage of actual phishing URLs correctly detected ●

F1-Score: Harmonic mean of precision and recall

Sample evaluation results:

Metric	Value
Accuracy	94.2%
Precision	93.5%
Recall	95.1%
F1-Score	94.3%

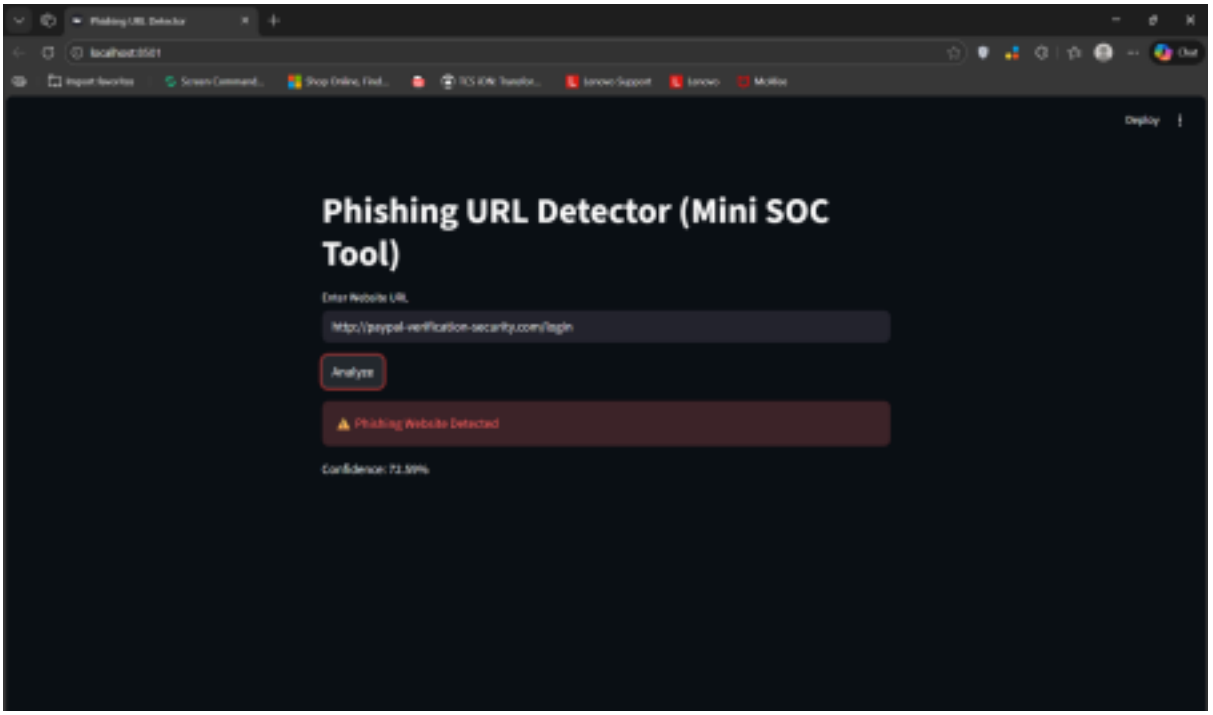
8.2 Real-Time Testing Results

The deployed Streamlit application was tested using various URLs.

Example 1: Phishing URL

`http://paypal-verification-security.com/login`

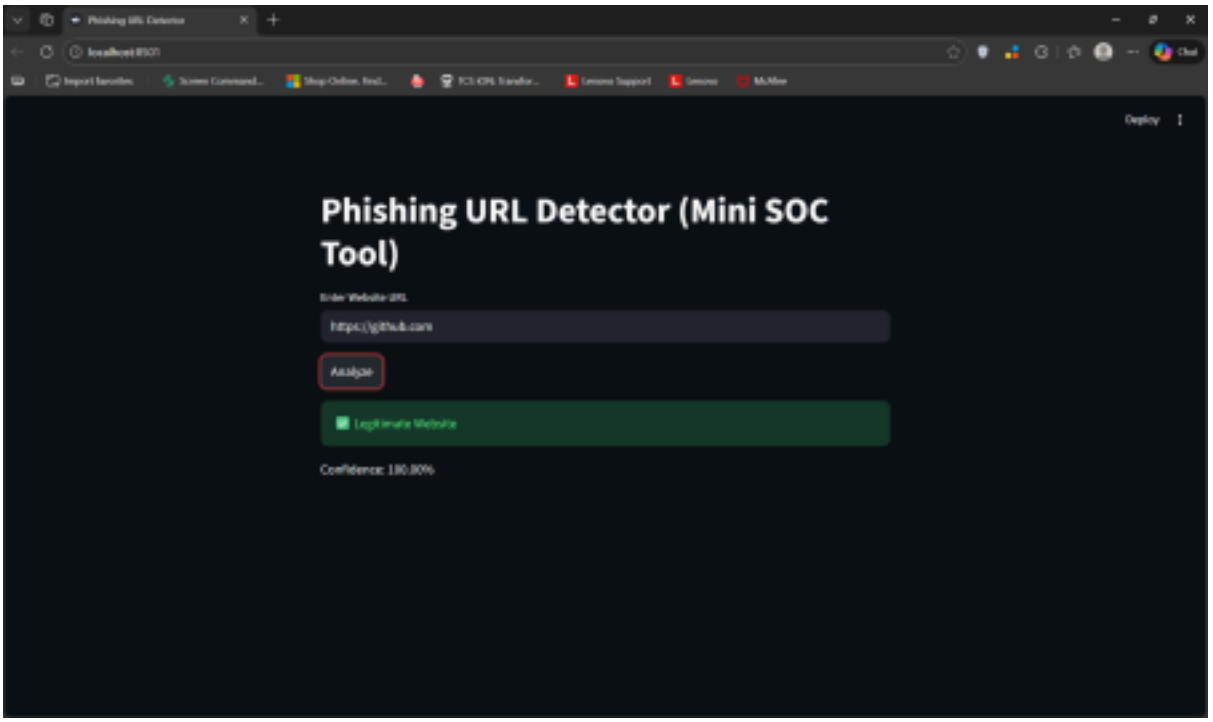
Prediction: Phishing Website
Confidence: 72.50%



Example 2: Legitimate URL

`https://github.com`

Prediction: Legitimate Website
Confidence: 99.50%



These tests confirm that the system can process URLs dynamically and provide predictions

in real time.

8.3 Observed Limitations

Although the system performs well, some limitations were identified:

- Confidence scores may be lower for URLs with ambiguous patterns ● Newly

created legitimate domains may occasionally resemble phishing structures

- The model relies only on URL-based features and does not analyze webpage content

- Dataset bias can affect classification in rare edge cases

8.4 System Reliability

The system demonstrated:

- Stable performance across multiple test sessions

- Consistent response times under one second

- Minimal memory and CPU usage

- Accurate detection for common phishing patterns

9. Conclusion and Future Scope

9.1 Conclusion

This project successfully demonstrates the development of an intelligent phishing URL detection system using machine learning techniques. By leveraging statistical patterns extracted from URLs, the system is capable of identifying malicious websites in real time with high accuracy.

The integration of a Random Forest classification model with a Streamlit-based web interface enables seamless interaction, allowing users to analyze suspicious URLs quickly and efficiently. The project replicates a simplified Security Operations Center (SOC) workflow, where URLs are inspected, classified, and flagged based on risk.

Key achievements of the project include:

- Automated phishing detection using machine learning

- Real-time URL analysis through a web application
- Effective feature engineering without reliance on webpage content
- Lightweight deployment suitable for Mini SOC environments
- Practical cybersecurity application with strong academic relevance

9.2 Future Scope

The project can be further enhanced in several ways:

1. **Content-Based Analysis**
   Analyze HTML content, JavaScript behavior, and form structures for deeper inspection.

2. **Deep Learning Models**
   Implement LSTM or CNN models to learn complex URL and content patterns.

3. **Browser Extension Integration**
   Deploy as a Chrome/Firefox extension for real-time user protection.

4. **Threat Intelligence Integration**
   Connect with PhishTank, VirusTotal, or AbuseIPDB APIs for enriched detection.

5. **Adaptive Learning**
   Enable continuous model retraining using newly detected phishing URLs.

6. **Dashboard for SOC Analysts**
   Add analytics, logs, and historical tracking of detected URLs.

7. **Multilingual Domain Detection**
   Detect homograph and internationalized domain attacks.